

Architect module report

All four module artifacts below (L2 map, L3 feature research, L4 refactor plan, L5 domain notes) analyze **honojs/hono**, a TypeScript web framework, at pinned commit `edd138ee3049749190de3dcd7e32d7bcbf224e41` (`git describe --tags: v4.13.7-gedd138ee`), cloned separately from this repository. The Builder application for this course round, **subscription-splitter** (Hono on Cloudflare Workers and D1), is a separate, freshly-built project and was not the analysis target for any of the four artifacts below.

1. Projects analyzed

Project	Stack	Scale	Used in
honojs/hono	TypeScript, zero runtime dependencies	~289 <code>src/</code> files, 2815 total commits, 76 public export subpaths	L2, L3, L4, L5
subscription-splitter (Builder app)	Hono, Cloudflare Workers, D1	New build, separate course project	Not analyzed here

2. Project map (L2)

Source: `context/map/repo-map.md`. All 76 public export subpaths funnel into a small, stable core (`types.ts` fan-in 53, `context.ts` fan-in 46 across 188 `src/` modules). That core is flagged as the top risk zone by three independent signals at once: git co-change, dependency-graph fan-in, and cycle membership. Layer boundaries (utils, router, middleware, adapter, client) are clean, zero violations found across 568 dependency edges. Core-spine knowledge is concentrated in one contributor, with a documented revert of a prior change there. Unknown: the dependency graph is scoped to `src/`; `runtime-tests/`, `perf-measures/`, and `benchmarks/` have no import graph, only git co-change evidence.

3. Feature analysis (L3)

Source: `context/changes/hono-request-dispatch-analysis/research.md`. Studied flow: request dispatch (Hono.fetch into `#dispatch`, router match, `compose()`, `Context`, `Response`), chosen because the map's risk zones and first-day list both point at this exact core. Feature overview: a request enters through `fetch()`, the path is matched against registered routes, a single matched handler runs directly while multiple handlers run through `compose()`, which advances a per-step route index as it walks the chain, and the resulting `Context` produces the final `Response`. Technical debt, at least one item ast-grep confirmed: `compose()`'s per-step route-index advancement has no test through a real multi-handler chain (confirmed: zero `matchResult` references in `compose.test.ts` against 30 `Context` constructions); the response's cookie-header-merge logic is implemented twice with opposite behavior, not in agreement as first assumed; the fast dispatch path lacks a safety guard the general dispatch path already has (confirmed as the guard's only occurrence in `src/`).

4. Refactor plan (L4)

Source: `context/changes/hono-refactor-opportunities/{research.md,plan.md}`, `context/decisions/D-004-hono-refactor-selection.md`. Originally selected: encapsulating a raw, public routing-state field behind an accessor. An independent plan review tested that choice with a counter-question and refuted three of its four supporting claims directly against the code (the field is documented for user reads, the cited precedent does not match its shape, and the historical bug it was meant to fix was actually caused elsewhere). The plan was re-ranked: that accessor is now deferred, and the selected changes are (1) a characterization test proving the route index advances correctly through a real multi-handler chain, and (2) adding the missing safety guard to the fast dispatch path, with its performance cost verified by the project's own benchmark rather than assumed free. Explicitly **not** doing: the deferred accessor (blocked, it would be a breaking change to a documented public property), normalizing the router's matching-result shape (a verified deliberate performance decision), or consolidating the cookie-merge duplication (needs its own characterization step first, since the two implementations were found to disagree, not just duplicate). Each phase is a separately reversible

commit with automated (typecheck, test, lint, format, and a bundle-size check where relevant) and manual (benchmark comparison against the pinned commit) verification.

5. Domain notes (L5)

Source: `context/domain/01-domain-distillation.md`, `02-invariant-aggregate-refactor.md`, `03-anti-corruption-layer.md`. Ubiquitous language: Hono, Context, Handler, Middleware, Router, runtime Adapter, and the "built on Web Standards" commitment; the sharpest model-vs-code gap is that the project's own README lists two runtimes (Node.js, Fastly Compute) that this repository's own adapters do not actually ship. Invariant #1: response cookies must be preserved when headers are merged, found not merely weakly tested but **actively violated today**, since the response's two header-merge code paths implement opposite cookie policies (one drops new cookies on reassignment, the other keeps both), independently corroborating the L4 finding. Anti-corruption layer: checked the AWS Lambda, Lambda@Edge, and Cloudflare Pages adapter boundaries plus the validation-library boundary; `grep/ast-grep` scoped outside each adapter directory found zero cross-boundary imports in every case, a genuine absent-leak result.

6. Decisions that belong to me

Choosing Hono, rather than the Builder application, as the module 4 analysis target was my call: the Builder app is a fresh build with no real commit history to mine, so analyzing it would have meant inventing contributor and co-change evidence that does not exist. Within Hono, I chose the request-dispatch flow specifically because two independent artifacts (map risk zones, first-day reading list) converged on it, not because it looked interesting. When the independent plan review refuted the top-ranked refactor candidate's supporting claims, I deferred that candidate rather than defending or patching the original justification, the corrected evidence changed what was actually worth doing, so the plan changed with it. Once the cookie-merge duplication turned out to be a behavioral disagreement rather than harmless repetition, I chose to defer consolidating it behind a characterization step rather than proposing a quick merge that could silently change one code path's behavior. For the anti-corruption-layer check, I chose to recommend a preventive CI check rather than any remediation, because the boundary was already clean; recommending a fix for a problem that does not exist would have been the wrong call.